

Buzzer

Hardware Overview

What is it?

The **Passive Buzzer** is a small sound-making component. It acts like a tiny speaker. You can control the **pitch** (frequency) to play different musical notes.

Electrical Explanation

Inside the housing is a **Piezoelectric element**. When you apply electricity to it, it deforms slightly.

- If you turn the electricity **ON** and **OFF** very fast, the element vibrates.
- These vibrations push air, creating sound waves.
- **Frequency:** The faster you switch it **ON/OFF**, the higher the pitch (e.g., 440 times a second = 440Hz = Note A4).

Key Hardware Facts:

- **Connection:** The buzzer is connected to **Digital Pin D9**.
- **Signal Flow:** Processor (Pin D9) → Transistor (Amplifier) → Buzzer → 0V.

(Note: The schematic uses a transistor (T1_Z) because the buzzer needs more current than the microcontroller pin can provide directly.)

Software & Programming

`kcomp_buzzer.h`

This library makes it easy to play single notes or melodies. **Usage:** `#include <kcomp_buzzer.h>`.

Function Reference

Function Name	Description & Parameters
<code>void initBuzzer()</code>	Setup: Configures Pin D9. Must be called in <code>setup()</code> .
<code>void buzzerBeepBlocking(uint freq)</code>	Manual Tone: Starts playing a tone at a frequency (Hz) and keeps playing it forever until you turn it off. ▪ <code>freq</code>: Frequency in Hertz (e.g., 440).
<code>void buzzerOff()</code>	Silence: Stops any currently playing tone immediately
<code>void buzzerBeepNonBlocking(uint freq, ulong duration)</code>	Timed Beep: Plays a tone for a set time, then stops. ▪ <code>freq</code>: Frequency in Hz. ▪ <code>duration</code>: Time in milliseconds.
<code>void buzzerMelodyBlocking(uint notes[], uint durations[], byte length)</code>	Play Song (Wait): Plays a full melody. The processor freezes until the song is done. notes: Array of note frequencies. durations: Array of note lengths (4=quarter, etc.).
<code>void buzzerMelodyNonBlocking(uint notes[], uint durations[], byte length)</code>	Play Song (Background): Plays a melody. The processor continues running your other code. (Requires <code>updateMelody()</code> in loop).

void updateMelody()

Checks if it's time to play the next note. **Must be called in loop()** if using non-blocking melodies.

Musical Notes

The library includes a list of standard musical notes.

- Examples: NOTE_C4 (Middle C), NOTE_FS5 (F# High).
- See kcomp_buzzer.h for the full list!

Sample Program

This program prints the direction and intensity to the Serial Monitor.

```
#include <kcomp_buzzer.h>

// --- Global Variables (Melody Data) ---
// We must define these outside setup/loop so they aren't deleted!
int melody[] = { NOTE_C4, NOTE_E4, NOTE_G4, NOTE_C5 };
int durations[] = { 8, 8, 8, 4 }; // 8=eighth note, 4=quarter note

void setup() {
  Serial.begin(115200);
  initBuzzer(); // Initialize Hardware

  // --- Demo 1: Manual Blocking Tone ---
  Serial.println("Test 1: Manual Beep (1 second)");
  buzzerBeepBlocking(NOTE_A4); // Turn sound ON (440Hz)
  delay(1000);                 // Wait 1s (Processor is blocked)
  buzzerOff();                 // Turn sound OFF manually
  delay(500);

  // --- Demo 2: Non-Blocking Sound Effect ---
  Serial.println("Test 2: Coin Sound (Non-Blocking)");
  // Play a high pitch for 200ms. Code continues IMMEDIATELY.
  buzzerBeepNonBlocking(NOTE_E6, 200);
  delay(1000); // We wait just so you can hear it clearly

  // --- Demo 3: Background Melody ---
  Serial.println("Test 3: Playing Melody in Background...");
  // Start the global melody (4 notes long)
  buzzerMelodyNonBlocking(melody, durations, 4); //
}

void loop() {
  // --- Essential: Keep the Music Playing ---
  // This function checks the clock and advances the melody if needed.
  updateMelody(); //

  // You can do other work here (like checking buttons or sensors)
  // and the music will keep playing automatically!
  delay(10);
}
```

 **Try it out:** You can find the full sample program **SimpleBuzzer** in the K-Comp examples folder!